

Chapter 1

L4 - The Storage Layer & Buffer Management

1.1 The storage hierarchy

A DBMS does not operate on one uniform memory. Data moves across a hierarchy of storage levels with very different latency, capacity, and cost.

- **CPU registers, caches, DRAM:** volatile, byte-addressable, and very fast.
- **SSD, HDD, network storage:** persistent, block-addressable, much larger, and much slower.
- Going *up* the hierarchy gives lower latency but less capacity and a higher cost per byte; going *down* gives the opposite.

Main memory holds the **working set**: the data currently being processed. Persistent storage holds the full database. Modern hardware adds intermediate levels such as **persistent memory** and **fast network storage**. Still, the DBMS must move pages carefully between storage and RAM.

Typical latency numbers

Operation	Latency	If 1 ns were 1 s
L1 cache reference	1 ns	1 s
L2 cache reference	4 ns	4 s
DRAM access	100 ns	100 s
SSD access	16,000 ns	4.4 h
HDD access	2,000,000 ns	3.3 weeks
Network storage	~ 50,000,000 ns	1.5 years
Tape archive	1,000,000,000 ns	31.7 years

The exact values vary by hardware, but the orders of magnitude do not: one extra I/O can cost more than millions of CPU instructions.

1.2 Why DBMSs still care about disks

A DBMS usually keeps only the actively used data in RAM and stores the database itself on secondary storage. This creates two expensive operations:

- **read**: transfer a page from disk to main memory,
- **write**: transfer a modified page from main memory back to disk.

Relative to in-memory operations, both are expensive, so the DBMS must plan I/O carefully.

1.2.1 Why not keep everything in main memory?

- **Cost**: large production databases can reach the TB or PB scale, so all-DRAM storage is expensive.
- **Volatility**: conventional DRAM loses data on power failure, but databases need persistence across runs.
- **Main-memory DBMSs**: useful for smaller or latency-critical workloads; larger memories and non-volatile devices make them increasingly practical.

For general-purpose systems, disks remain the default place to store data.

1.2.2 Pages and storage devices

Data is stored and retrieved in units called **pages** (or **disk blocks**), not byte by byte.

Page (disk block). The basic unit of transfer between secondary storage and main memory. DBMS I/O is organized around whole pages.

Access cost depends on both the device and the access pattern:

- **magnetic disks (HDDs)**: very large capacity and low cost per byte, but slow random access; good streaming throughput.
- **flash storage (SSDs)**: lower latency and strong random-read performance, but still far slower than DRAM and more constrained on writes.
- **layout matters**: unlike RAM, page access time depends heavily on where a page is placed and whether access is sequential or random.

1.3 Magnetic disk organization

A magnetic disk stores bits on rotating platters coated with magnetic material. Each disk surface is organized geometrically.

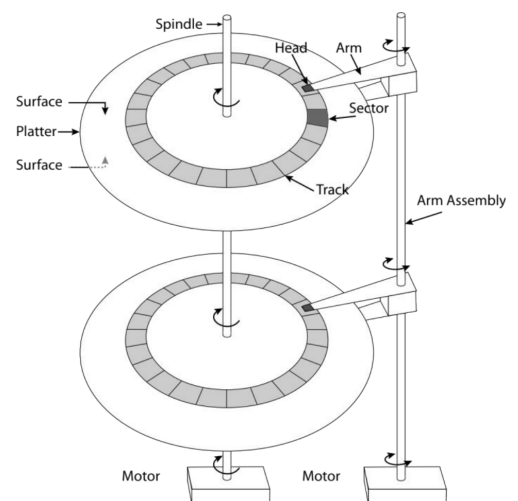
Track. A concentric circle on one disk surface.

Sector. A slice of a track and the smallest hardware-addressable unit on the disk.

Cylinder. The set of tracks at the same radius across all disk surfaces.

Important details:

- outer tracks are longer, so they can store more sectors and offer higher transfer bandwidth than inner tracks,
- disks are divided into **zones**: all tracks in one zone have the same number of sectors per track,
- the hardware exposes a sector-addressable space; common sector sizes are 512 B or 4096 B,
- a DBMS page size is a fixed multiple of the sector size,
- the main operations are **read** and **write**.



1.3.1 Cost of accessing a page

For a magnetic disk, page access time is well approximated by

$$\text{access time} = \text{seek time} + \text{rotational delay} + \text{transfer time}.$$

Seek time. Time to move the disk arm so the head reaches the correct track. Typically, 5–15 ms.

Rotational delay. Waiting time until the desired sector rotates under the read/write head. At 7200 RPM, one rotation takes about 8.3 ms, so the average delay is about half of that.

Transfer time. Time to actually move the bytes once the head is already in position. For one page, this is usually much smaller than seek and rotation.

This is why **sequential access** is much faster than random access: once the head is already near the right place, the DBMS can read neighboring blocks with little extra positioning cost.

1.3.2 Arranging pages on disk

To minimize latency, blocks of the same file should be laid out in a sensible physical order. A good notion of the **next** block is:

- first, a block on the same track,
- then, a block on the same cylinder,
- then, a block on an adjacent cylinder.

Pre-fetching. Reading nearby pages before they are explicitly requested, because sequential layout makes them cheap to fetch once the disk is already positioned.

Physical placement matters a lot: a disk block has several useful physical neighbors, not just one logical successor.

1.3.3 A simple disk-performance model

Assume a magnetic disk with:

- average seek time: 5 ms,
- spindle speed: 7200 RPM $\Rightarrow$ one rotation in $\frac{60,000}{7200} \approx 8.33$ ms, so average rotational delay ≈ 4.17 ms,
- transfer rate: 50 MB/s,
- page size: 4096 B $\Rightarrow$ transfer time = $\frac{4096}{50 \times 10^6}$ s ≈ 0.082 ms.

Example: 4 KB reads on a 7200 RPM HDD

Access pattern	Time	Throughput
Random block anywhere on disk	$5 + 4 + 0.082 = 9.082$ ms	≈ 451 KB/s
Random block in the same cylinder	$4 + 0.082 = 4.082$ ms	≈ 1.03 MB/s
Next block on the same track (sequential)	0.082 ms	≈ 50 MB/s

The gap is enormous: good physical layout and sequential scans can improve effective bandwidth by two orders of magnitude. Page placement and clustering matter. Buffer management builds on exactly this gap.

1.3.4 Rules of thumb

Two heuristics explain most storage-level design decisions:

- main-memory access is roughly $\sim 1000\times$ faster than disk I/O,
- on magnetic disks, sequential I/O is roughly $\sim 10\times$ faster than random I/O.

Thus, DBMSs try to keep hot data in memory and lay out pages sequentially on secondary storage.

1.4 Flash storage

Flash memory has existed for decades and evolved from USB keys and mobile-device storage into consumer and enterprise **SSDs**. In DBMSs, it can serve either as **main storage** or as an **accelerator** such as a cache or logging device.

1.4.1 Why flash changed storage

Flash disks are one of the main modern solutions for fast storage:

- they offer orders-of-magnitude higher I/O performance than HDDs,
- they expose the same block-oriented API as disks, so DBMSs can still work with pages,
- there is no mechanical seek, so **random reads** are close in cost to **sequential reads**,
- unlike HDDs, access time is not determined by physical page location on the device.

Flash page and flash block. Reads and writes operate at **page** granularity, while erases operate at **block** granularity. A flash block contains many pages.

1.4.2 Internal organization

An SSD is a small parallel system rather than a single chip:

- a controller runs firmware and exports a disk-like block interface,
- flash packages are connected through **channels**,
- each package contains several **dies** (chips),
- each die contains **planes**, **blocks**, and **pages**.

This is why flash access time depends on controller design, internal parallelism, firmware, and package bandwidth, not on a moving disk head. In typical devices, a block may contain dozens of 4 KB pages.

1.4.3 Operations, density, and lifetime

Flash supports three basic operations:

- **read** and **write** at page granularity,
- **write** can only program cells from $1 \rightarrow 0$,
- **erase** resets cells from $0 \rightarrow 1$, but only at block granularity.

Common flash families

Type	Bits/cell	Typical trade-off
SLC	1	fastest, longest lifetime, highest cost
MLC	2	more capacity, lower endurance
TLC	3	cheaper per bit, lower endurance and slower writes
QLC	4	highest density, lowest endurance; best for read-heavy use

As density rises from **SLC** to **QLC**, cost per bit falls, but endurance and write performance usually decline. Device lifetime often ranges roughly from 10^5 down to 10^3 **program/erase (P/E) cycles**.

Wear leveling. Firmware techniques that spread writes and erases across blocks so that no small region of flash wears out too early.

1.4.4 Flash Translation Layer (FTL)

Flash Translation Layer (FTL). The firmware layer that gives SSDs an HDD-like interface. It remaps logical pages to physical flash locations, performs garbage collection, applies wear leveling, and tunes both performance and device lifetime.

The FTL is necessary because flash cannot overwrite pages in place the way magnetic disks can.

1.4.5 Flash versus HDDs

In practice, the two technologies serve different goals:

- **HDDs:** best when the main goal is **large, inexpensive capacity**.
- **Flash:** best when the workload needs **low latency, high IOPS**, and especially **efficient random reads**.

1.5 Storage requirements and RAID

Storage systems should ideally be **fast, reliable, and affordable**. A single disk rarely optimizes all three at once.

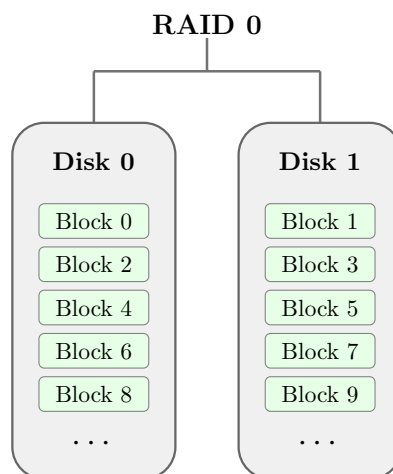
RAID. Redundant Array of Inexpensive Disks. RAID builds one logical disk from several physical disks to improve throughput, reliability, or both.

RAID combines two main ideas:

- **striping:** spread data across disks for higher throughput,
- **redundancy:** duplicate or protect data so the array can survive failures.

1.5.1 RAID 0: striping

- file blocks are striped across disks,
- there is **no redundancy**, so RAID 0 provides **no fault tolerance**,
- throughput is excellent because the array can use the cumulative bandwidth of all disks,
- usable capacity is the sum of the capacities of all disks.

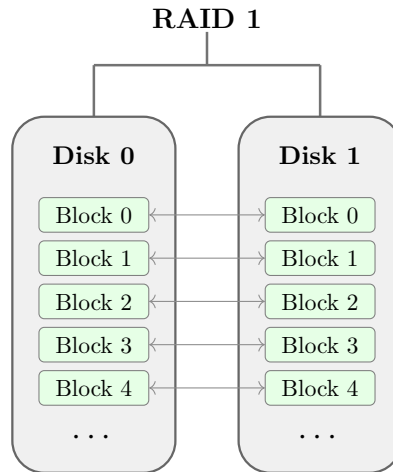


The downside is reliability: with N independent disks, the mean time to failure of the striped array is roughly $\frac{1}{N}$ of the mean time to failure of one disk, because *any* disk failure breaks the array.

1.5.2 RAID 1: mirroring

- every block is duplicated on another disk,
- RAID 1 handles **disk loss** well, but it does **not** protect against logical corruption or bad writes mirrored to both copies,
- usable capacity is only that of one disk per mirror pair,
- reads can be parallelized, but each write must update all mirrored copies.

RAID 1 is simple and reliable, but expensive: half of the raw capacity is used for redundancy.



1.5.3 RAID 5: striping with rotating parity

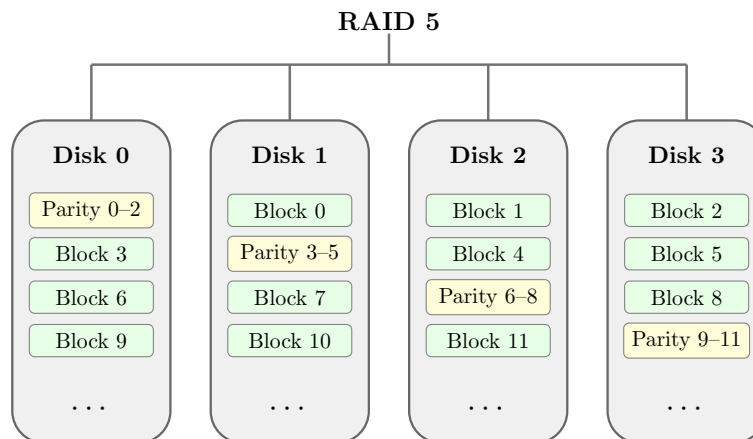
Parity. An extra block computed as the bitwise XOR of several data blocks. If one block is missing, it can be reconstructed from the parity block and the surviving data blocks.

For one stripe group,

$$P = S_0 \oplus S_1 \oplus \dots \oplus S_{N-2}.$$

- data is striped across disks, but parity blocks rotate from disk to disk,
- the array can survive the loss of **one** disk,
- usable capacity is the equivalent of $N - 1$ disks out of N ,
- reads are fast, but writes are more complex because parity must be updated.

RAID 5 is a common compromise: it is more affordable than mirroring, more reliable than pure striping, and widely used in servers and datacenters.



1.6 Disk space management

Disk space management. The lowest storage-oriented layer of the DBMS. It manages free space on disk. It also exports page-level operations such as allocate, deallocate, read, and write.

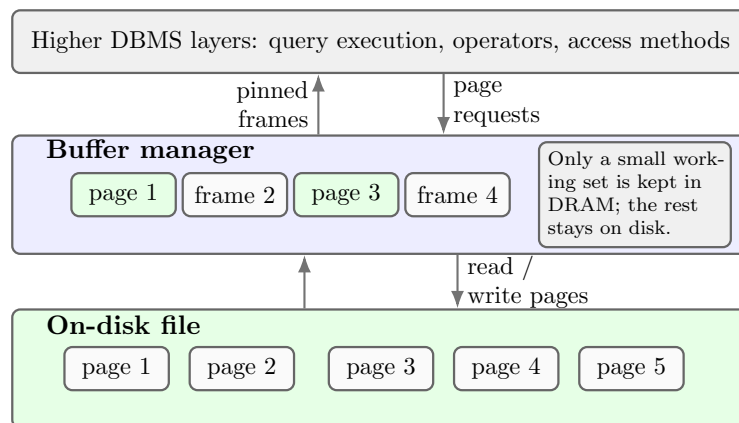
Higher layers should not have to manage physical layout directly. They request pages, while this layer decides where pages live on disk and how free space is tracked.

The best case is when a request for a **sequence of pages** is satisfied by pages stored sequentially on disk. Then the DBMS gets the full benefit of fast sequential I/O without exposing storage details to upper layers.

In the DBMS stack, **buffer management** sits directly above disk space management. Higher levels ask for pages in memory, and the buffer manager turns those requests into page I/O.

1.7 Buffer management

A DBMS can only process data that is currently in DRAM. The buffer manager hides the fact that most pages still live on disk, much like a hardware cache hides the gap between CPU and main memory.



1.7.1 Why DBMSs do not rely only on the OS

The operating system already provides a file system and a block cache, but DBMSs still want to manage pages their own way:

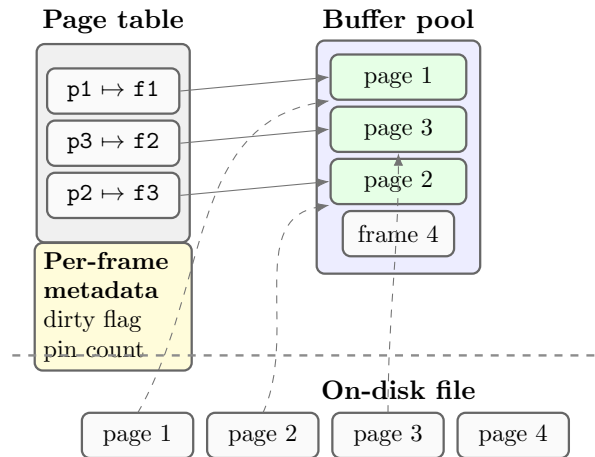
- **custom prefetching:** many database scans are predictable, so the DBMS can fetch pages ahead of time,
- **replacement control:** the DBMS wants to choose which pages stay in memory instead of accepting a generic OS policy,
- **flushing control:** recovery protocols such as **WAL** may require log records to reach disk before dirty data pages are written,
- **scheduling control:** OS scheduling can interfere with DBMS locking and create **convoy effects**, where one blocked worker delays many others.

1.7.2 Buffer pool, frames, and metadata

Buffer pool. A region of DRAM organized as an array of fixed-size slots used to cache database pages.

Frame. One slot in the buffer pool. When the DBMS fetches a page, an exact copy of that page is placed into one frame.

Page table. An in-memory mapping from page IDs to the frames that currently hold them. It exists only in memory and is not stored on disk.



For each cached page, the buffer manager also stores metadata:

- **dirty flag**: whether the in-memory copy was modified and must be written back before eviction,
- **pin count** (reference count): how many DBMS components are currently using the page,
- **replacement metadata**: for example timestamps, access counters, or reference bits.

Dirty pages are often kept in memory for a while instead of being written back immediately. This avoids turning every update into a synchronous disk stall.

1.7.3 Handling a page request

Pinned page. A page whose pin count is greater than zero because some operator or transaction is actively using it. Pinned pages are not valid eviction candidates.

When higher layers request a page:

- on a **hit**, the page is already in the buffer pool, so the DBMS returns its frame and increments the pin count,
- on a **miss**, the DBMS must first find a frame:
 - if an empty frame exists, use it,
 - otherwise choose a victim according to the replacement policy,
 - only pages with pin count = 0 are candidates.
- if the chosen victim is **dirty**, write it back to disk first,
- read the requested page into the chosen frame,
- pin the page and return its address to the caller.

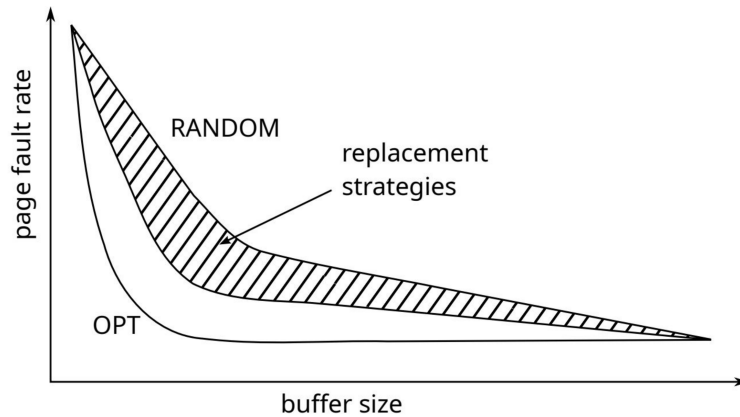
When the caller finishes, it must **unpin** the page and indicate whether the page was modified. A page may be pinned several times, so it becomes evictable only when its pin count returns to 0.

If the request pattern is predictable, for example during a long sequential scan, the buffer manager can **pre-fetch** several pages at once. Replacement can also trigger extra I/O for recovery, because **write-ahead logging** may force older log records to disk before a dirty data page is flushed.

1.7.4 Replacement policies

When the buffer pool is full, some cached page must be replaced:

- a **clean** page can be discarded immediately,
- a **dirty** page must first be written back,
- the policy should balance correctness, hit rate, speed, and metadata overhead.



Common policies include **FIFO**, **LRU**, **MFU**, **Clock**, **LRU-K**, and **2Q**. In practice, DBMSs prefer policies that are both effective and cheap to maintain online.

FIFO (first-in-first-out)

- keep frames in queue order,
- insert newly loaded pages at the tail and evict from the head,
- very simple, but it ignores locality: an old page may still be hot and yet be removed first.

FIFO is easy to implement, but it does **not** reliably retain frequently reused pages.

LRU (least recently used)

- evict the unpinned page whose last access is the oldest,
- conceptually this means keeping a timestamp per page,
- a practical implementation keeps frames in recency order, often with a doubly-linked list,
- frequently reused pages move toward the back and tend to stay cached.

LRU is one of the most widely used policies because it exploits temporal locality well.

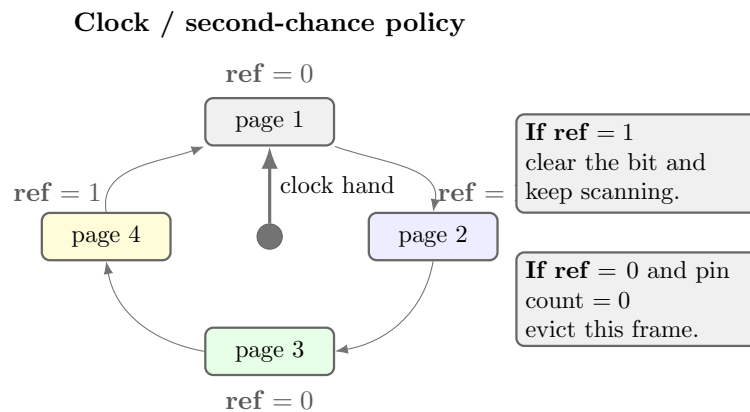
Clock (second chance)

Clock policy. An approximation of LRU that stores one **reference bit** per frame instead of a full access timestamp.

The policy organizes frames conceptually in a circle with a moving **clock hand**. Each access sets the page's reference bit to 1. When a victim is needed:

- if the current frame is pinned, skip it,
- if its reference bit is 1, clear the bit and advance,
- if its reference bit is 0, evict it.

A hit does not move the page to a different frame; it only turns the reference bit back on. This makes Clock cheaper than exact LRU while keeping much of the same intuition.

**MFU**

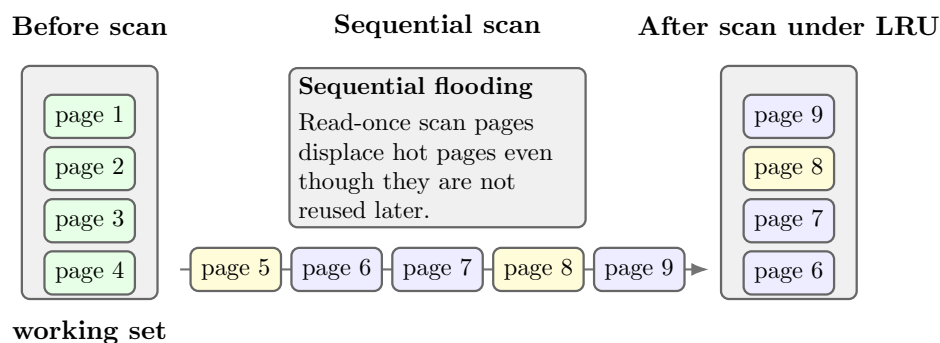
- **MFU** (most frequently used) uses access frequency rather than recency,
- it is less common in practice, but it shows that reuse history can be richer than “last access only”.

Sequential flooding

Scan resistance. The ability of a replacement policy to avoid polluting the buffer pool with pages that are touched once during a large scan and then never reused.

Both **LRU** and **Clock** can suffer from **sequential flooding**. A large sequential scan may push useful pages out of memory even though the scanned pages are not part of the long-term working set.

- the query reads pages $p_5, p_6, \dots$ one by one,
- each scanned page causes a miss and briefly enters the pool,
- if $\#frames < \#pages$ in the file, then almost every request triggers an I/O,
- the buffer pool ends up filled with read-once pages, while previously hot pages have been evicted.



This is why modern policies try to distinguish **reuse** from a one-time access.

LRU-K

LRU-K. An extension of LRU that tracks the last K references to each page and uses this history to estimate whether the page is likely to be reused.

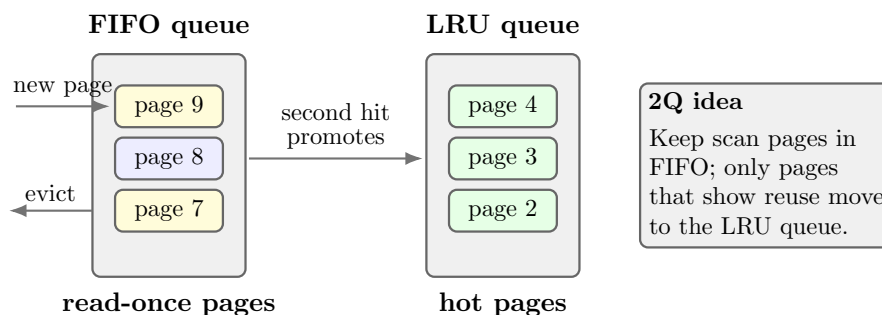
- pages with fewer than K recorded accesses are **cold**,
- pages with at least K accesses are **warm**,
- cold pages are evicted before warm pages,
- if all candidates are warm, compare the K -th most recent access and evict the page whose K -th access is the oldest,
- when $K = 1$, LRU-K reduces to ordinary LRU.

Because one-off scan pages usually remain **cold**, LRU-K is much more resistant to sequential flooding.

2Q

2Q. A scan-resistant policy that keeps two queues: a FIFO queue for recently seen pages and an LRU queue for pages that proved they are reused.

- new pages first enter the **FIFO** queue,
- if a page is referenced again while it is still known to the system, it is promoted to the **LRU** queue,
- evictions prefer the FIFO queue, so read-once scan pages are discarded there,
- hot pages stay in the LRU queue and are protected from scan pollution.



In short: **read-once pages stay in FIFO, reused pages graduate to LRU.** This makes 2Q simple, practical, and much more scan resistant than plain LRU or Clock.